Sistema de distribucion de habitantes del reino constitucional de Therania

Proyecto final POO

Juan David Moreno Ramírez//20242005116

Samuel Andrés Rojas Escobar//20252020077

2026-1

THERANIA

COMMIT:

HASH:

Índice

- **Índice** 3
- **Introducción** 4
- **Objetivos** 5
- **Cronograma de Trabajo** 6
- **Manual de usuario** 9
- **Casos de uso** 10
- **Javadoc** 12
- **Diagramas UML** 13
- **Uso de IA** 14
- **Resultados** 16

Introducción

El presente proyecto, titulado Sistema de distribución de habitantes del reino constitucional de Therania, corresponde al desarrollo de una aplicación orientada a objetos elaborada como proyecto final de Programación Orientada a Objetos en Java 17. El sistema fue diseñado con el propósito de modelar de manera estructurada y funcional la organización social del reino ficticio de Therania, permitiendo la gestión de ciudadanos, especies auto-percibidas, manadas, rituales y diferentes procesos relacionados con la interacción entre estas entidades. Durante el desarrollo se aplicaron principios fundamentales de la Programación Orientada a Objetos, tales como herencia, encapsulamiento, composición y polimorfismo, buscando construir una arquitectura clara, escalable y coherente con la complejidad del dominio planteado. Además, el sistema incorpora persistencia de datos mediante archivos JSON, simulación automática de información, generación de reportes e implementación de una interfaz gráfica que facilita la interacción del usuario con las funcionalidades disponibles.

El proyecto no solo se enfocó en el desarrollo técnico del software, sino también en la documentación y organización del proceso de construcción. Por ello, el documento incluye objetivos, cronograma de trabajo, manual de usuario, casos de uso, diagramas UML, Javadoc y una reflexión ética sobre el uso de herramientas de inteligencia artificial durante el desarrollo. Todo esto permite evidenciar tanto las decisiones de diseño tomadas como el aprendizaje adquirido a lo largo del proyecto.

Objetivos

1. Diseñar e implementar un sistema orientado a objetos que modele fielmente el dominio de Theriania Construir una arquitectura en Java 17 que represente correctamente las entidades del problema —ciudadanos, especies, manadas y rituales— haciendo uso explícito de herencia, polimorfismo y composición, de modo que el diseño refleje la complejidad del dominio sin reducirlo a soluciones simplistas como campos de texto plano.

2. Desarrollar un sistema funcional con persistencia, simulación e interfaz de usuario Implementar todas las funcionalidades mínimas requeridas: registro y consulta de ciudadanos, cálculo del Índice de Afinidad Animal, generación de reportes y ranking del Alfa Honorario, modo simulación con datos aleatorios, y una interfaz gráfica clara y usable, respaldada por una capa de persistencia en archivos JSON.

3. Producir documentación técnica completa que evidencie el proceso y las decisiones de diseño Generar los entregables requeridos —diagramas UML, Javadoc, manual de usuario, cronograma y reflexión sobre uso ético de IA— de forma que cualquier persona pueda entender el sistema, reproducirlo y evaluar las decisiones tomadas durante el desarrollo.

Cronograma de Trabajo

El Proyecto se desarrolló en diferentes etapas y dias, las fechas exactas y lo realizado en dichas fechas:

04/23/2026

- Commit inicial, creación del proyecto.
- Primeras clases del reino de theriana.
- Creación de carpetas de manadas e imagenes.
- Primer boseto de diagrama UML.
- Interfez grafica inicial.

04/24/2026

- Ciudadano therian V1 y AfiliacionMnada V1.
- Adición de mapas, funciones de rastreo de mouse y retoques graficos.

04/26/2026

- Adición de primeros iconos de especie.

04/28/2026

- Adición de iconos totales de especie.

04/30/2026

- Actualización de especies y ciudadanos.
- Therians V3.

05/02/2026

- Adición de clase .ºpcion 5.eiimplementación de tablas de datos e inicio de gestión de Archivos JSON.
- Clase documentos para manejo de archivos (escritura, lectura y guardado) pasando como parametros la información de las clases presentes en src (de preferencia usando arraylist).

05/04/2026

- Documentación del proyecto PDF

05/07/2026

- Especies
- Corrección REQ1
- Servicios
- REQ 4 y 5
- REQ 6
- REQ 7 y 8
- REQ 3
- Reportes
- Therian V1

05/14/2026

- Corrección de errores de ejecución e implementación directa de GUI
- Lectura de archivos en GUI

- Therian V2
- Therian V3
- Therian V4

05/15/2026

- Therian V5
- Therian V6
- Sin JSON

05/16/2026

- Gestión documental del proyecto
- Desarrollo total de diagrama UML

Manual de usuario

title

Casos de uso

Gestión de ciudadanos

- Registrar ciudadano therian: el sistema permite crear un nuevo ciudadano con sus datos civiles básicos (nombre, identificador único y fecha de nacimiento) y asignarle un estado de ciudadanía inicial.

- Consultar ciudadano: permite buscar y visualizar la información completa de un ciudadano, incluyendo su historial de especies auto-percibidas y sus afiliaciones a manadas.

- Actualizar estado de ciudadanía: el administrador puede modificar el estado de un ciudadano entre activa, suspendida o en revisión según corresponda.

Gestión de especies auto-percibidas

- Registrar especie auto-percibida: permite asociar a un ciudadano una especie de la jerarquía disponible (Lobo, Felino, Ave, etc), registrando la fecha de inicio de dicha identidad.

- Consultar historial de especies: muestra la trayectoria completa de especies que un ciudadano ha declarado a lo largo del tiempo, con fechas de inicio y cierre de cada una.

Gestión de manadas

- Afiliar ciudadano a manada: registra el ingreso de un ciudadano a una manada mediante una instancia de AfiliacionManada, especificando su rol (alfa, beta, omega u observador) y su nivel de compromiso declarado.

- Listar manadas activas: muestra el listado completo de manadas con su número de miembros actuales y la especie predominante de cada una.

- Consultar miembros de manada: presenta el detalle de todos los ciudadanos afiliados a una manada específica, incluyendo sus roles y fechas de ingreso.

Gestión de rituales

- Registrar participación en ritual: asocia a un ciudadano como participante de un ritual existente, actualizando automáticamente su indicador de afiliación efectiva.
- Consultar historial de rituales: lista todos los rituales en los que ha participado un ciudadano o todos los llevados a cabo por una manada, con sus detalles completos.

Cálculo y reportes

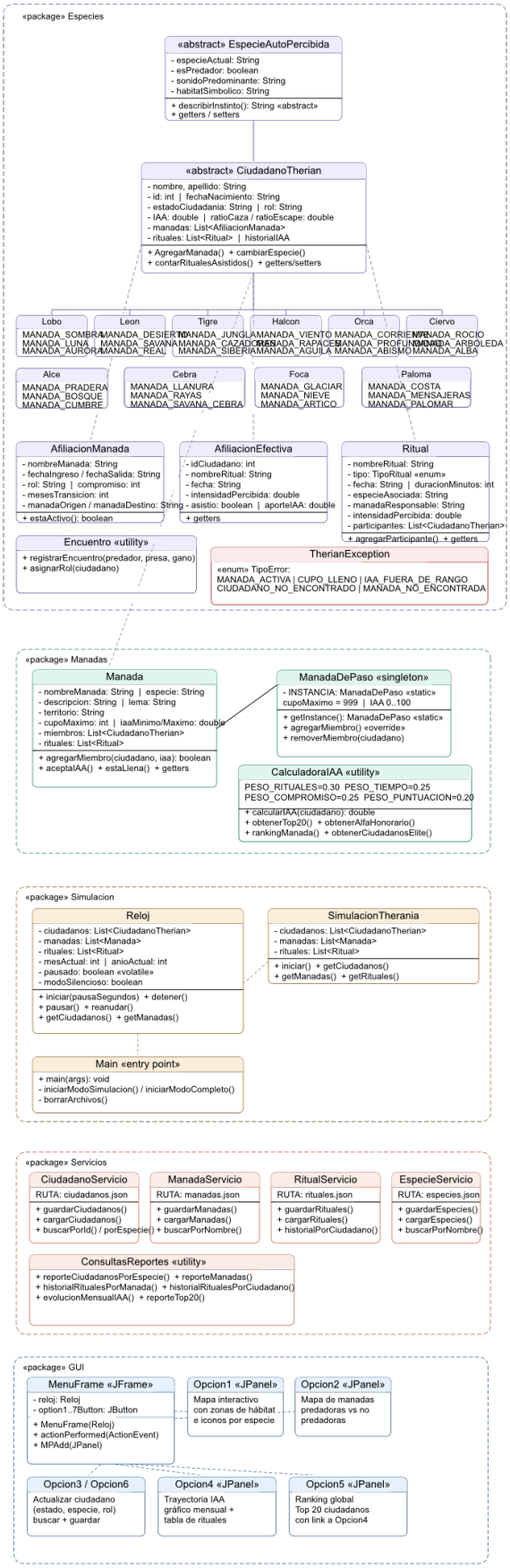
- Calcular Índice de Afinidad Animal (IAA): ejecuta la fórmula definida por el equipo sobre los datos de un ciudadano (rituales asistidos, estabilidad de especie, nivel de compromiso y feedback de manada) para obtener su IAA actualizado.
- Generar ranking del Alfa Honorario: produce el listado de los 20 ciudadanos con mayor IAA global, mostrando su especie actual, manada principal y resumen de historial ritual.
- Ver evolución mensual del IAA: muestra la progresión del índice de afinidad de un ciudadano mes a mes mediante una estructura de puntos (mes, IAA) apta para generación de gráficos.

Simulación

- Activar modo simulación: genera automáticamente un conjunto de datos de prueba compuesto por al menos 100 ciudadanos, 10 manadas activas, 4 tipos de especie autopercibida y 500 rituales distribuidos a lo largo de un año, sin intervención manual.
- Cargar / guardar datos JSON: permite serializar el estado completo del sistema en archivos JSON estructurados y deserializarlo en sesiones posteriores, garantizando la persistencia de toda la información entre ejecuciones.

Javadoc

Diagramas UML



Uso de IA

Reflexión sobre el uso ético de la IA en el desarrollo del proyecto

Durante el desarrollo de este proyecto recurrimos a herramientas de inteligencia artificial, específicamente ChatGPT y GitHub Copilot, en distintos momentos del proceso. Sin embargo, desde el principio tuvimos claro que su papel sería el de un apoyo puntual, no el de un reemplazo de nuestro propio criterio. Esta reflexión busca ser honesta sobre cómo las usamos, para qué sirvieron y dónde decidimos no depender de ellas.

¿Para qué las usamos?

GitHub Copilot fue útil principalmente en las partes más repetitivas del código: generación de getters y setters, estructuras de bucles conocidas, y autocompletado de bloques que ya sabíamos cómo debían quedar. En esos casos, Copilot aceleró la escritura, pero la decisión de qué escribir y por qué siempre fue nuestra. Nunca aceptamos una sugerencia sin leerla y entenderla primero.

ChatGPT lo usamos más como una herramienta de consulta cuando nos atascamos en problemas puntuales: errores de serialización en JSON que no lográbamos rastrear, dudas sobre el comportamiento de ciertas clases de Java, o para entender mejor algún concepto antes de implementarlo. La dinámica era siempre la misma: nosotros describíamos el problema con precisión, evaluábamos la respuesta, y decidíamos si tenía sentido aplicarla al contexto específico de nuestro sistema. Muchas veces la respuesta nos daba una pista, pero la solución final la construíamos nosotros.

Lo que no le delegamos a la IA

El diseño orientado a objetos fue completamente nuestro. La jerarquía de especies, la relación entre ciudadanos y manadas, la lógica del Índice de Afinidad Animal y la arquitectura en

capas del sistema son decisiones que tomamos nosotros, con base en lo aprendido en clase y en las restricciones del enunciado. Pedirle a una IA que diseñara eso por nosotros habría vaciado de sentido el ejercicio, y además habríamos obtenido algo que no entenderíamos ni podríamos defender en la sustentación.

Tampoco usamos IA para generar bloques de código que luego pegáramos sin más. Cuando Copilot sugería algo que no comprendíamos del todo, lo descartábamos o lo investigábamos por nuestra cuenta antes de considerarlo. El código que está en el repositorio lo escribimos, lo leímos y lo probamos nosotros.

Qué aprendimos de este proceso

Usar IA de forma responsable no significa no usarla, sino saber cuándo y para qué. En un proyecto académico como este, donde el objetivo es aprender a diseñar y programar, depender de la IA para las decisiones importantes habría sido contraproducente: quizás habríamos entregado algo que funciona, pero no habríamos desarrollado el criterio que se supone que debemos adquirir.

Lo que sí nos pareció valioso fue usarla como se usa cualquier otra herramienta: con un propósito claro, sin perder el hilo de lo que estamos construyendo ni por qué lo estamos construyendo así. Al final, la IA nos ahorró tiempo en cosas menores para poder dedicar más energía a lo que realmente importaba: entender el problema y resolverlo bien.

Resultados